Homework 3: Weakest Preconditions

```
use vstd::prelude::*;

verus! {
    mod problem_1 {
        use vstd::prelude::*;

        fn m(x: isize, y: isize) -> (result: (isize, isize))
            requires
                x != y,
            ensures
                result.0 > result.1
        {
            if x > y {
                (x, y)
            } else {
                (y, x)
            }
        }

        proof fn m_gp(x: isize, y: isize)
        {
            // Use conditional (if) rule
            let ghost branch_then: bool = (x > y);
            let ghost branch_else: bool = (y > x);
            let ghost condition_rule: bool = ((x > y) ==> branch_th
            let ghost req: bool = x != y;
            assert(req ==> condition_rule);
        }
    }

    mod problem_2 {
        use vstd::prelude::*;

        exec fn m(x0: isize) -> (x: isize)
            requires
                usize::MIN <= x0 - 3 <= usize::MAX
            ensures
                (x0 < 3 ==> x == 1),
                (x0 >= 3 ==> x < x0),
        {
            let mut x = x0 - 3;
            if x < 0 {
                x = 1;
            } else {
                if true {
                    x = x + 1;
                } else {
                    x = 10;
                }
            }
            x
        }
```

```
    }

    proof fn m_wp(x0: isize)
    {
        let ghost x: int = x0 - 3;
        let ghost postcondition: bool = (x0 < 3 ==> x == 1) ||
        let ghost inner_then: bool = (true && x0 == x0 + 1);
        let ghost inner_else: bool = x0 == 10;
        let ghost outer_then: bool =  (x0 < 3 && x == 1);
        let ghost outer_else: bool = (x0 >= 3 && (inner_then ||
        let ghost condition_rule: bool = outer_then && outer_el
        assert(condition_rule ==> postcondition);
        let ghost req: bool = usize::MIN <= x0 - 3 <= usize::MA
        assert(req ==> postcondition);
    }
}

mod problem_3 {
    use vstd::prelude::*;

    spec fn is_median_spec(a: int, b: int, c: int, m: int) -> (
        ||| (m == a && (b <= a <= c || c <= a <= b))
        ||| (m == b && (a <= b <= c || c <= b <= a))
        ||| (m == c && (a <= c <= b || b <= c <= a))
    }

    spec fn median_spec(a: int, b: int, c: int) -> (int) {
        if (a >= b && a <= c) || (a >= c && a <= b) {
            a
        } else if (b >= a && b <= c) || (b >= c && b <= a) {
            b
        } else {
            c
        }
    }

    proof fn lemma_median_spec_is_median(a: int, b: int, c: int
        requires
            m == median_spec(a, b, c),
        ensures
            is_median_spec(a, b, c, m),
    {
        let ghost branch_then = (a >= b && a <= c) || (a >= c &&
        let ghost branch_else_if = (b >= a && b <= c) || (b >=
        let ghost branch_else = !(branch_then || branch_else_if

        if branch_then {
            assert(m == a);
            assert((a >= b && a <= c) || (a >= c && a <= b));
            assert(is_median_spec(a, b, c, m));
        } else if branch_else_if {
            assert(m == b);
            assert((b >= a && b <= c) || (b >= c && b <= a));
```

```
                assert((b >= a && b <= c) || (b >= c && b <= a));
                assert(is_median_spec(a, b, c, m));
            } else {
                assert(m == c);
                assert((c >= a && c <= b) || (c >= b && c <= a));
                assert(is_median_spec(a, b, c, m));
            }
        }
    }

    fn main() {}
} // verus!
```